Technische Hochschule Deggendorf

MDM-M-2 Case Study Intelligent Systems in Production

Binary Classification of AOI Alarms into True Defects and False Calls

Second check of AOI results with supervised machine learning
(Siemens production data)

Bibliography Report

Team 1

Yu Han
Melisa Subasi
Meron Hagos
Alejandro Rivas

Under the supervision of:
Prof. Dr.-Ing. Hamidreza Heidari

Date of submission: 03.11.2026

Abstract

In electronics assembly, automated optical inspection (AOI) checks every solder joint after soldering and sends every flagged board to a manual inspection station. Most of these alarms are false calls: in public Siemens production data, about 99 % of the flagged solder joints are false calls. A second check with supervised machine learning after the AOI could release many false calls without manual inspection, and Siemens is working on this problem in its Erlangen electronics factory today.

This report reviews 36 sources, from the general to the specific. Earlier studies show that classifiers such as random forests can recognise false calls from AOI measurements. On the same Siemens data, however, models that meet the targets on a test set from the same period (at most 1 % missed defects, at least 40 % fewer manual checks) miss 5–15 % of the true defects on every later production period. The literature on dataset shift and concept drift, cases from manufacturing, and the authors of the data and of the study themselves point to drift, a change of the data over time, as the cause; an unstable method for setting the decision threshold adds to it. The standard remedy in the drift literature is to retrain the model regularly on recent data. In related applications, a binary fraud classifier with less than 1 % positives and a quality model on a semiconductor materials line, regular retraining reduced the loss in performance. For the second check of AOI results, this remedy has not been tested in the literature we reviewed.

Our project therefore repeats the experiment of Pfab and Rothering on the same data with the same models and methods, and addresses the two causes they name, one at a time: the threshold is set correctly for the final model, and the model is retrained during production. The deliverable is a simulation of the second check in production, with retraining and the threshold strategy as switches; its results are compared with the published result, and the targets of the original study (at most 1 % missed true defects, at least 40 % fewer manual checks) serve as reference lines.

Contents

Abstract **i**

1 Introduction **1**

1.1 Objectives 1

1.2 Scope of the review 1

2 Materials and Methods **1**

2.1 Method of the literature search 1

2.2 Reviewed literature 1

2.2.1 From zero-defect manufacturing to AOI in electronics assembly 2

2.2.2 False calls of AOI: the Siemens data and earlier studies 3

2.2.3 Classifiers for tabular data 4

2.2.4 Evaluation with rare true defects 6

2.2.5 Why models fail over time: drift 6

2.2.6 Methods for the first next step: setting the threshold 8

2.2.7 Methods for the second next step: handling drift 8

2.3 Summary of the reviewed works 10

3 Conclusion **10**

References **12**

List of Figures

1 Reviewed literature from general to specific (numbers refer to the reference list). 2

2 Missed true defects reported by Pfab and Rothering [8] (Tables IV and V, mean ± SD of their runs): on target on the test set, far above it on all later slices. 4

List of Tables

1 Studies on AOI false calls compared with our project. 10

2 Classifier families for tabular data: evidence and role in our project. 10

3 Chain of evidence behind our research question. 11

4 From the literature to the switches of our simulation (the two next steps named in [8]). . 11

5 Methods of our project and the sources they are based on. 12

1 Introduction

Zero-defect manufacturing relies on detecting defects before a product leaves the factory [1]. In electronics assembly, components are soldered onto printed circuit boards by surface-mount technology (SMT), and AOI is placed after soldering [2]. AOI compares camera measurements of each solder joint with fixed limits; every flagged board goes to a manual inspection station, where an operator decides whether it is a true defect or a false call [5]. Many flagged boards are false calls, which creates a large manual workload [5, 6]. A classifier after the AOI that releases false calls without missing true defects would reduce this workload.

1.1 Objectives

- To review the literature on AOI false calls and on supervised classifiers used as a second check of AOI results.
- To identify why such classifiers fail on later production days, and what the literature offers against it.
- To justify the design of our project: a one-to-one repetition of the study of Pfab and Rothering [8] that changes one factor at a time: the threshold, then retraining.

1.2 Scope of the review

The review covers 36 sources published between 1996 and 2026: peer-reviewed journal and conference papers, one data article, one technical report, two preprints, one workshop paper and one industry press release. It is limited to AOI of solder joints in SMT assembly and to tabular AOI measurements; image-based defect detection with deep learning is outside the scope, because our project starts after the AOI machine has made its measurements.

2 Materials and Methods

2.1 Method of the literature search

The sources were searched in five topics, ordered from general to specific: AOI and false calls; classifiers for tabular data; evaluation with rare classes; causes of failure over time; and retraining as a remedy. Each source was read and summarised, and every reference was checked against its DOI or official page. For the central study [8], the full text and the published source code and run logs were read.

- **Databases:** IEEE Xplore, ScienceDirect, SpringerLink, ACM Digital Library, arXiv, Google Scholar, Crossref, Mendeley Data, GitLab and the Siemens press room.
- **Search terms:** *automated optical inspection*, *false call*, *SMT*, *solder joint*, *tabular data benchmark*, *imbalanced data*, *ROC analysis*, *dataset shift*, *concept drift*, *manufacturing*, *retraining*, *sliding window*.
- **Inclusion criteria:** the source deals with AOI false calls or the Siemens AOI data, with a method we use, or with model failure and retraining under drift; it is peer-reviewed or a primary source; its full reference can be verified. Preprints and the workshop paper are marked as such.
- **Reading:** for each source we noted what was done, the data, the main result and how it relates to our project. Numbers are only taken from the sources or from our own result files.

2.2 Reviewed literature

Figure 1 shows how the sources are ordered. The following subsections follow this order.

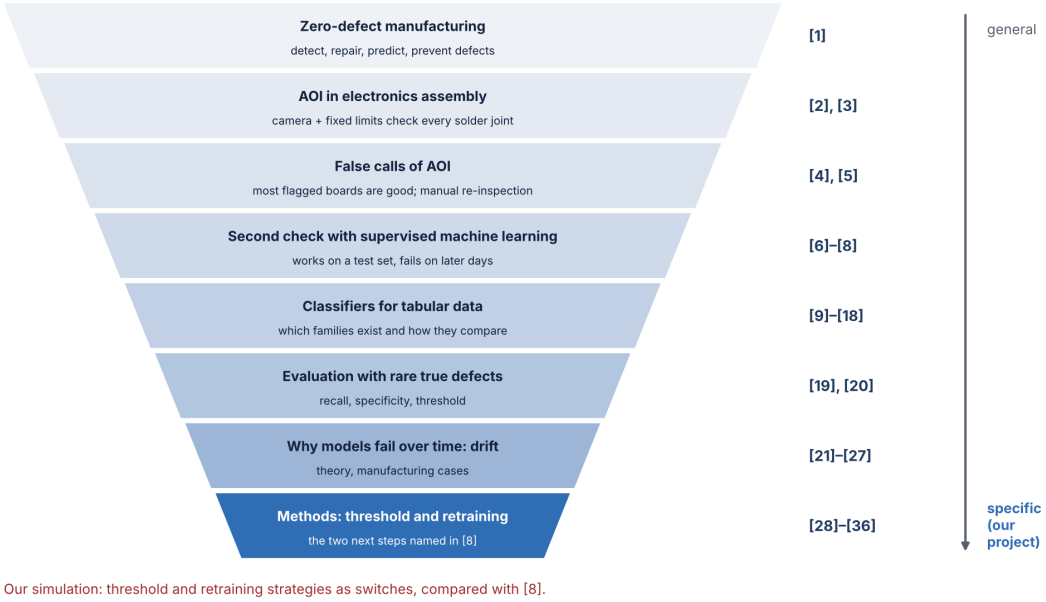


Figure 1: Reviewed literature from general to specific (numbers refer to the reference list).

2.2.1 From zero-defect manufacturing to AOI in electronics assembly

Zero defect manufacturing: state-of-the-art review
F. Psarommatis et al. (2020), Int. J. Production Research [1]

The authors review 280 articles on zero-defect manufacturing published between 1987 and 2018 and group the approaches into four strategies: detection, repair, prediction and prevention of defects.

Relevance to our project: AOI and the manual inspection station belong to the *detection* strategy. A second check makes detection cheaper without letting more defects pass.

A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry
A. A. R. M. Abu Ebayyeh and A. Mousavi (2020), IEEE Access [2]

This review describes AOI systems in electronics production: the hardware (cameras and lighting) and the software steps (pre-processing, feature extraction, classification). AOI replaces human visual inspection, which is slow and suffers from fatigue.

Relevance to our project: It explains where our input comes from: AOI measures each solder joint and compares it with fixed limits; the second check works on these measurements.

Short review on machine learning optimization methods in surface mounted electronics assembly technologies
P. Martinek et al. (2023), ISSE [3]

This short review collects machine learning applications along the SMT line, such as tuning stencil printing and component placement. It states that much research has used machine learning, for example neural networks, to improve AOI.

Relevance to our project: It places our project in an established field: machine learning at the inspection step of the SMT line.

Siemens and NVIDIA advance AI-powered PCB inspection from digital twin to factory
Siemens (Oct. 2026), press release [4]

Siemens and NVIDIA prepare AOI programs on the computer before production; the solution is shown in the Siemens electronics factory in Erlangen. When the AOI flags a possible defect, a second inspection step decides whether it is a real defect or a false call. The stated targets are up to 85 % less manual engineering effort for AOI programs, 40 % fewer false calls and more than 5 % higher first-pass yield.

Relevance to our project: The false-call problem is a current industrial problem. The 40 % is a target, not a published result.

2.2.2 False calls of AOI: the Siemens data and earlier studies

Data of automated optical inspection of surface-mounted technology electronic production
K. Pfab et al. (2024), Data in Brief [5]

The authors publish 132 days of AOI data from one SMT line of Siemens AG in Germany. Every row is a solder joint that the AOI flagged, with its AOI measurements and the label from the manual inspection station: true defect or false call. The authors warn that the share of wrong labels is unknown, because people set them, and that "the data may contain distribution drifts of unknown type".

Relevance to our project: This is our data set. The authors themselves name drift as a property of the data.

A machine learning based approach to detect false calls in SMT manufacturing
N. Thielen et al. (2020), ISSE [6]

The authors test whether a machine learning step after the AOI can recognise false calls from the numerical AOI measurements. They compare k-nearest neighbours, a neural network and random forests and vary the data set size and the decision threshold, aiming to recognise as many false calls as possible without more missed defects (slip). k-nearest neighbours works best on small data sets but becomes too slow on large ones, where neural networks and random forests perform better.

Relevance to our project: Same task on the same kind of data. The data are not public, and whether the test data come after the training data in time is not reported.

Enhancing false call detection in automated optical inspection using XGBoost classifier and XAI algorithms
J. Albayari et al. (FAIM 2025, publ. 2026), Springer LNME [7]

The authors use AOI data from Siemens AG and compare XGBoost, random forest, a deep neural network and k-nearest neighbours as a second check after the AOI, with several train–test splits. XGBoost gives the best balance of recall and precision; LIME and SHAP show which AOI measurements drive the predictions.

Relevance to our project: Same classifier types on Siemens data; no evaluation on later production periods is reported.

Towards improved research methodologies for industrial AI: a case study of false call reduction
K. Pfab and M. Rothering (2025), arXiv preprint [8]

This is the central study for our project. On the data of [5], the authors split the rows in time order: the first half is used for modelling (a stratified random 80/20 split into a tuning set and a test set), the second half is cut into five later slices. Their goal is at most 1 % missed true defects (slip) and at least 40 % fewer

manual checks (volume reduction). They tune k-nearest neighbours, random forests, balanced random forests, XGBoost and an AutoML model with Bayesian optimisation and 5-fold cross-validation; the decision threshold is the mean of the thresholds found in the five folds, and each model is trained once. Two models, a random forest and a balanced random forest, meet the goal on the test set, but on all five later slices they miss 5–15 % of the true defects (Figure 2); the authors conclude that the model "would fail from the first moment in production". A principal component analysis shows data clusters that "did just appear in a certain period of time", and the authors state that "various distribution drifts" occur in the data. As possible next steps they name "improving the method for setting the threshold and handling the distribution drifts", without testing either. Neural networks are not tested.

Relevance to our project: Our project repeats this experiment one-to-one and adds retraining. Their published code and run logs make an exact comparison possible.

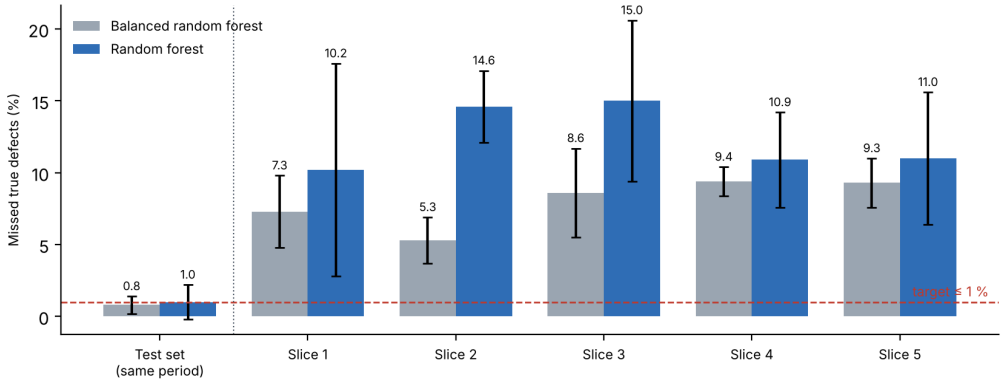


Figure 2: Missed true defects reported by Pfab and Rothering [8] (Tables IV and V, mean ± SD of their runs): on target on the test set, far above it on all later slices.

2.2.3 Classifiers for tabular data

AOI measurements are tabular data. Large benchmark studies show which classifier families work on such data, and the original papers describe the models used in [8].

Do we need hundreds of classifiers to solve real world classification problems?
M. Fernández-Delgado et al. (2014), J. Machine Learning Research [9]

The authors evaluate 179 classifiers from 17 families on 121 data sets. Random forests are the best family; a support vector machine with Gaussian kernel is close behind without a significant difference.

Relevance to our project: No single classifier wins everywhere, but random forests are a strong default for tabular data.

Why do tree-based models still outperform deep learning on typical tabular data?
L. Grinsztajn et al. (2022), NeurIPS Datasets and Benchmarks [10]

On 45 tabular data sets, tree-based models (XGBoost, random forests) remain state of the art for medium-sized data, even without counting their speed advantage.

Relevance to our project: Supports tree ensembles as the main models and shows why deep tabular models are not needed for our question.

Tabular data: deep learning is not all you need
R. Shwartz-Ziv and A. Armon (2022), Information Fusion [11]

XGBoost outperforms recent deep learning models on tabular data, including the data sets used in the papers that proposed these models, and needs much less tuning.

Relevance to our project: Same conclusion from an independent comparison.

Deep neural networks and tabular data: a survey
V. Borisov et al. (2024), IEEE Trans. Neural Networks and Learning Systems [12]

The survey compares traditional methods with 11 deep learning approaches and finds that gradient-boosted tree ensembles still mostly outperform deep learning on supervised tasks.

Relevance to our project: Third independent confirmation for tree ensembles on tabular data.

When do neural nets outperform boosted trees on tabular data?
D. McElfresh et al. (2023), NeurIPS Datasets and Benchmarks [13]

On 176 data sets with 19 algorithms, the difference between gradient-boosted trees and neural networks is often negligible, and light tuning of the trees matters more than the choice of family; boosted trees cope better with skewed or heavy-tailed features.

Relevance to our project: The choice of model family is less important than often assumed; this supports keeping the models of [8] fixed and changing only the operation.

Accurate predictions on small data with a tabular foundation model
N. Hollmann et al. (2025), Nature [14]

TabPFN, a model pre-trained on synthetic data, outperforms tuned baselines on data sets with up to 10,000 samples.

Relevance to our project: Our training sets have more than 100,000 rows, outside the range of this model.

Random forests
L. Breiman (2001), Machine Learning [15]

A random forest is a set of decision trees, each trained on a random sample of the data with a random subset of features at each split; the trees vote. Random forests reach an accuracy comparable to boosting, are robust to noise and give a measure of variable importance.

Relevance to our project: One of the two models of [8] that we repeat.

Using random forest to learn imbalanced data
C. Chen, A. Liaw and L. Breiman (2004), technical report [16]

The report proposes two variants for rare classes: a balanced random forest, which draws an equal number of cases from each class for every tree, and a weighted random forest.

Relevance to our project: The second model of [8] that we repeat; true defects are about 1 % of the flagged joints.

XGBoost: a scalable tree boosting system
T. Chen and C. Guestrin (2016), ACM SIGKDD [17]

The paper introduces XGBoost, a fast and scalable implementation of gradient-boosted decision trees.

Relevance to our project: Used in [7, 8]; gradient boosting is one of the families in our planned extension.

Scikit-learn: machine learning in Python
F. Pedregosa et al. (2011), J. Machine Learning Research [18]

The paper introduces scikit-learn, a Python library with a consistent interface for standard machine learning methods and tools for pre-processing and evaluation.

Relevance to our project: All models and data preparation in our project use this library.

2.2.4 Evaluation with rare true defects

Learning from imbalanced data
H. He and E. A. Garcia (2009), IEEE Trans. Knowledge and Data Engineering [19]

The review explains why accuracy is misleading when one class is rare, presents class weighting and resampling, and recommends class-wise metrics such as recall and specificity.

Relevance to our project: We report slip ($1 - \text{recall}$) and volume reduction (specificity) instead of accuracy, as [8] does.

An introduction to ROC analysis
T. Fawcett (2006), Pattern Recognition Letters [20]

A classifier that gives a score is turned into a decision by a threshold; a lower threshold catches more positives but gives more false alarms.

Relevance to our project: The 1 % slip target is met by choosing the threshold; the threshold therefore sits at the extreme end of the score distribution of the true defects, where few cases decide its value.

2.2.5 Why models fail over time: drift

A unifying view on dataset shift in classification
J. G. Moreno-Torres et al. (2012), Pattern Recognition [21]

"Dataset shift appears when training and test joint distributions are different." The authors distinguish covariate shift (the inputs change), prior probability shift (the class shares change) and concept shift (the relation between inputs and class changes). One main cause is a non-stationary environment, a "temporal difference between training and test data"; the shift often has a negative effect on classifier performance.

Relevance to our project: It gives the terms and the cause we use: the AOI data of later periods may differ from the training data in the measurements, in the share of true defects, or in the relation between both.

A survey on concept drift adaptation
J. Gama et al. (2014), ACM Computing Surveys [22]

Concept drift is a change of the joint distribution $p(X, y)$ over time. The survey groups adaptation into memory (which data to keep or forget), change detection and learning; one learning mode is "retraining that discards the current model and builds a new model from scratch". A fixed sliding window that periodically retrains on the latest examples is "often used as a baseline".

Relevance to our project: Periodic retraining on recent data is the baseline remedy for drift; this is what we add to [8].

Learning under concept drift: a review
J. Lu et al. (2019), IEEE Trans. Knowledge and Data Engineering [23]

The review explains that "the induced pattern of past data may not be relevant to the new data" and names as the most direct reaction to "retrain a new model with the latest data to replace the obsolete model", usually with a window of the most recent data.

Relevance to our project: Second standard reference for retraining as the remedy.

Handling data drift in deep learning-based quality monitoring
J. C. Bauer et al. (2025), J. Intelligent Manufacturing [24]

For the optical quality monitoring of friction stir welds (good or not good), the authors test the same models on normal data and on data with controlled drift: other tool geometries, alloys and sheet thicknesses, more brightness and blur. All metrics drop on the drifted data.

Relevance to our project: A controlled manufacturing experiment showing that drift alone lowers the performance of a quality classifier.

Adaptive fault detection framework for recipe transition in semiconductor manufacturing
J. Shim et al. (2021), Computers & Industrial Engineering [25]

A fault detection model for an etch process, trained on one recipe, loses much of its performance when the recipe changes, because the sensor data distribution changes; the authors adapt the model to the new recipe.

Relevance to our project: A production example in which a change of the process causes the classifier to fail.

On the reliability of machine learning applications in manufacturing environments
N. Jourdan et al. (2021), NeurIPS workshop [26]

On a sensor data set recorded over 36 months, "the accuracy strongly declines with increasing drift for all tested algorithms"; the authors conclude that "none of the tested algorithms is robust against drifts" and that monitoring and model updates are required.

Relevance to our project: Changing the algorithm alone does not solve drift; the operation of the model has to change.

Review of adaptation mechanisms for data-driven soft sensors
P. Kadlec et al. (2011), Computers & Chemical Engineering [27]

For data-driven models in the process industry, the review organises adaptation methods by concept drift theory into moving-window techniques, recursive adaptation and ensembles.

Relevance to our project: Retraining on a moving window is an established practice for industrial models.

2.2.6 Methods for the first next step: setting the threshold

Pfab and Rothering name "improving the method for setting the threshold" as a next step [8]. The literature offers three methods.

Neyman-Pearson classification algorithms and NP receiver operating characteristics
X. Tong, Y. Feng and J. J. Li (2018), Science Advances [28]

When one type of error must stay below a bound α, the common practice of limiting the *empirical* error to α "does not satisfy the [...] error control objective", because the resulting classifiers "are still likely to have [...] errors much larger than α". The authors' umbrella algorithm, which works with any scoring classifier such as random forests, takes the threshold as an order statistic of the scores of a left-out sample and so guarantees that the error stays below α with probability at least $1-\delta$. This needs a minimum size of the left-out sample, $n \geq \log\delta / \log(1-\alpha)$.

Relevance to our project: Our bound is at most 1 % missed true defects. With $\alpha = 0.01$ and $\delta = 0.05$, the left-out data must contain at least 299 true defects; with fewer, no threshold can guarantee the bound. This explains why a threshold for 1 % slip is unstable, and gives a method to set it with a stated confidence.

Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure
M. Saerens et al. (2002), Neural Computation [29]

When the share of a class in use differs from its share in training, the classifier's outputs no longer fit; the authors re-estimate the new class shares from new, unlabelled data and adjust the outputs.

Relevance to our project: If the share of true defects on the line changes, a threshold fixed once cannot follow; it has to be set again for the current period.

Detecting and correcting for label shift with black box predictors
Z. C. Lipton, Y.-X. Wang and A. Smola (2018), ICML [30]

Under label shift the class shares $p(y)$ change while $p(x \mid y)$ stays the same. The authors estimate the new class shares from the confusion matrix of any trained classifier, without labels of the new data, propose a test to detect the shift, and correct the classifier with the estimated shares.

Relevance to our project: After the second check is in use, released joints are no longer inspected and have no label; this method detects a change of the defect share without these labels.

2.2.7 Methods for the second next step: handling drift

The second next step named by Pfab and Rothering is "handling the distribution drifts within the dataset" [8]. Retraining is the basic remedy [22, 23, 27]; the methods below decide when to retrain and on which data.

Learning in the presence of concept drift and hidden contexts
G. Widmer and M. Kubat (1996), Machine Learning [31]

The authors learn from a window of currently trusted, recent examples, store concept descriptions to reuse them when an earlier context returns, and use a heuristic that monitors performance to control both.

Relevance to our project: The origin of window-based retraining: the training data are the most recent examples, not all history.

Learning with drift detection
J. Gama et al. (2004), SBIA [32]

The drift detection method (DDM) tracks the online error rate of the model. A warning level is reached when the error plus its standard deviation exceeds the lowest value seen by two standard deviations, and a drift level at three; at a drift, a new model is learned from the examples since the warning.

Relevance to our project: An alternative to retraining on a fixed schedule: retrain only when the error rises.

Learning from time-changing data with adaptive windowing
A. Bifet and R. Gavaldà (2007), SIAM SDM [33]

ADWIN keeps a window of recent data whose size is "recomputed online according to the rate of change observed from the data in the window itself", with formal bounds on false and missed detections.

Relevance to our project: An alternative to a fixed training window: the window adapts to how fast the line changes.

Out-of-sample tests of forecasting accuracy: an analysis and review
L. J. Tashman (2000), Int. J. Forecasting [34]

The review recommends tests with a rolling origin: the model is fitted on all data up to a point in time, tested on the next period, and the origin moves forward.

Relevance to our project: This is how every retraining method is evaluated in our simulation: before each period, only data up to that point are used.

Credit card fraud detection: a realistic modeling and a novel learning strategy
A. Dal Pozzolo et al. (2018), IEEE Trans. Neural Networks and Learning Systems [35]

Fraud detection is a binary task with fewer than 1 % positives on a stream of more than 75 million transactions over three years. The classifier is updated every day. A classifier trained on the most recent days ranks frauds better than one trained on data a week older, which the authors take as evidence that the stream is non-stationary and that recent data are more informative; their strategy that uses the newest labelled transactions performs best throughout.

Relevance to our project: The closest classification analogue to our task: binary, rare positives, a long daily stream, and regular retraining that helps.

Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification
M. Gao et al. (2026), arXiv preprint [36]

On five years of data from a semiconductor materials line, a random forest that predicts product quality (regression) degrades after a supplier change and with equipment wear. Retraining every 100 batches gives "only marginal improvements"; retraining every 5 batches gives about 80 % fewer out-of-range predictions. The retraining frequency matters more than the choice between a fixed and a growing training window, and new hyper-parameter tuning at each retraining is not worth its cost. The authors prefer a fixed schedule to drift-triggered retraining, which "will only be triggered after poor predictions have been observed".

Relevance to our project: A manufacturing case in which retraining works; the frequency is the main setting. Differences: regression instead of classification, and a preprint.

2.3 Summary of the reviewed works

Table 1 compares the studies on AOI false calls. Table 2 summarises the classifier families. Table 3 shows the chain of evidence behind our project, Table 4 maps the methods from the literature to the switches of our simulation, and Table 5 lists the methods of our project and their sources.

Table 1: Studies on AOI false calls compared with our project.

	Data	Classifiers	Evaluation	Training
Thielen et al. [6]	not public	kNN, neural network, random forest	time order not stated	no retraining reported
Albayari et al. [7]	Siemens AG	XGBoost, random forest, DNN, kNN	several train–test splits	no retraining reported
Pfab & Rothering [8]	Siemens, public [5]	kNN, RF, balanced RF, XGBoost, AutoML	test set + 5 later slices	once
Our project	Siemens, public [5]	same as [8] (RF, balanced RF)	same as [8]	**once vs. retrained before each slice**

Table 2: Classifier families for tabular data: evidence and role in our project.

Family	Evidence	Role in our project
Random forest, balanced RF	best family on 121 data sets [9]; best models in [6, 8]	core: repeated one-to-one from [8]
Gradient boosting (XGBoost)	strongest on tabular data in [10–12]; best in [7]	extension
Neural network (multi-layer perceptron)	often close to boosted trees [13]; not tested in [8]	extension
Linear (logistic regression)	simple baseline	extension
k-nearest neighbours	too slow on large data [6]; weak in [8]	not used
Deep tabular models, TabPFN	not better than trees [10–12]; TabPFN only up to 10,000 samples [14]	not used

3 Conclusion

The literature leads from a general goal to one specific question. Zero-defect manufacturing needs reliable defect detection [1]; in electronics assembly this is done by AOI [2, 3], which raises many false calls that operators check by hand [5], a problem that industry is still working on [4]. Supervised classifiers after the AOI can recognise false calls [6, 7], and on tabular data like the AOI measurements tree ensembles are the strongest family [9–12].

On the same public data that our project uses, however, Pfab and Rothering [8] show that models which meet the targets on a test set from the same period miss 5–15 % of the true defects on every later period. The evidence points to drift as the main cause: by definition, a model fails when the data it meets differ from the data it learned from [21–23]; controlled and real manufacturing cases show this effect [24–26]; and for the Siemens data the authors themselves warn of drift and show data clusters that appear only in

Table 3: Chain of evidence behind our research question.

Claim	Evidence	Sources
1. A second check trained once fails on later production periods	on target on the test set, 5–15 % missed defects on all five later slices	[8]
2. Two causes: drift and the threshold method	definition and effect of dataset shift; drift lowers quality classifiers in manufacturing; drift is named for this data by its authors and shown by a PCA in [8]; an empirical threshold at a 1 % error bound does not hold with few cases	[5, 8, 21–26, 28]
3. Methods exist for both	threshold: order statistic with a stated confidence, re-setting under changed class shares; drift: periodic, window-based, drift-triggered or adaptive-window retraining	[22, 23, 27–33]
4. Retraining helps in related applications	binary fraud classifier with rare positives; semiconductor materials line	[35, 36]
5. Named as next steps for AOI false calls, not tested	"improving the method for setting the threshold and handling the distribution drifts"	[8]

Table 4: From the literature to the switches of our simulation (the two next steps named in [8]).

Switch	Option	What it does	Sources
Threshold	as in [8]	mean of the five cross-validation thresholds	[8]
	order statistic with confidence	1 % bound held with probability $1 - \delta$; needs enough left-out true defects	[28]
	re-set for the current period	follows a changed share of true defects, also without labels	[29, 30]
Retraining	off	train once, as in [8]	[8]
	on a fixed schedule	per period, weekly, daily; growing or sliding window	[22, 31, 35, 36]
	triggered by drift	retrain when the error rises, or adapt the window	[32, 33]

certain periods [5, 8]. The method for setting the threshold, named by [8] as a second problem, makes it worse: a threshold set on the empirical error does not keep a 1 % bound unless enough true defects are left out for it [28], and a threshold fixed once does not follow a changing class share [29, 30].

The standard remedy for drift is to retrain the model regularly on recent data [22, 23, 27]. It has helped in a binary classification task with rare positives [35] and on a semiconductor materials line, where the retraining frequency mattered most [36]. For the second check of AOI results, the reviewed literature does not report whether it works.

Position of our project. In their discussion, Pfab and Rothering name as "potential next steps to stabilize the modeling" the "improving [of] the method for setting the threshold and handling the distribution drifts within the dataset" [8], without testing either. Our project focuses on exactly these two steps. We do not propose a new method. We test whether the standard remedies work for this real problem: we repeat the experiment of [8] one-to-one, on the same data, with the same models, hyper-parameters and threshold method, and change one factor at a time for the two causes named in [8]: first the threshold, set correctly for the final model, then retraining before each later production period. The question is: how do missed defects and released false calls change when the second check runs in production with retraining and threshold re-setting, compared with training once? The project delivers a production simulation in which retraining and the threshold strategy can be switched, and every result is shown next to the published result of [8]; its targets (at most 1 % missed true defects, at least 40 % fewer manual checks) serve as

Table 5: Methods of our project and the sources they are based on.

Method	Reason	Sources
Binary classification of each flagged joint	the manual inspection label has two classes	[5, 8]
Same data split, models, hyper-parameters and threshold method as [8]	one-to-one comparison	[8, 15, 16]
Threshold strategy as a switch	first next step named in [8]	[8, 28–30]
Retraining strategy as a switch, evaluated with a rolling origin	second next step named in [8]	[22, 31–36]
Slip and volume reduction; reference lines $\leq$1 % and $\geq$40 %	business metrics and targets of [8]	[8, 19, 20]
Ten random seeds, mean $\pm$ SD	as in [8]	[8]
Extension: more model families	choice of family matters less than often assumed	[10, 13]

reference lines. Whatever the result, it answers a question that the original study left open. Before the experiment, we will check the data for drift with the definitions of [21]; the results, and the extension to retraining frequency and further model families, follow in the final report.

References

[1] F. Psarommatis, G. May, P.-A. Dreyfus, and D. Kiritsis, "Zero defect manufacturing: state-of-the-art review, shortcomings and future directions in research," *Int. J. Production Research*, vol. 58, no. 1, pp. 1–17, 2020, doi: 10.1080/00207543.2019.1605228.

[2] A. A. R. M. Abu Ebayyeh and A. Mousavi, "A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry," *IEEE Access*, vol. 8, pp. 183192–183271, 2020, doi: 10.1109/ACCESS.2020.3029127.

[3] P. Martinek, I. M. Putrama, O. Krammer, and A. Géczy, "Short review on machine learning optimization methods in surface mounted electronics assembly technologies," in *Proc. 46th Int. Spring Seminar on Electronics Technology (ISSE)*, 2023, pp. 1–6, doi: 10.1109/ISSE57496.2023.10168524.

[4] Siemens, "Siemens and NVIDIA advance AI-powered PCB inspection from Digital Twin to factory," press release, Plano, TX, USA, Oct. 21, 2026. [Online]. Available: `https://news.siemens.com/en-us/siemens-nvidia-pcba-digital-twin-factory/`

[5] K. Pfab, R. Eichler, A. Mallandur, and M. Rothering, "Data of automated optical inspection of surface-mounted technology electronic production," *Data in Brief*, vol. 53, Art. no. 110110, Apr. 2024, doi: 10.1016/j.dib.2024.110110. Data: Mendeley Data, v2, doi: 10.17632/99jzmh9658.2.

[6] N. Thielen, D. Werner, K. Schmidt, R. Seidel, A. Reinhardt, and J. Franke, "A machine learning based approach to detect false calls in SMT manufacturing," in *Proc. 43rd Int. Spring Seminar on Electronics Technology (ISSE)*, Demanovska Valley, Slovakia, 2020, pp. 1–6, doi: 10.1109/ISSE49702.2020.9121044.

[7] J. Albayari, P. Patra, D. Patel, and D. Santos, "Enhancing false call detection in automated optical inspection using XGBoost classifier and XAI algorithms," in *Flexible Automation and Intelligent Manufacturing (FAIM 2025)*, Lecture Notes in Mechanical Engineering. Cham, Switzerland: Springer, 2026, pp. 509–530, doi: 10.1007/978-3-032-05610-8_51.

[8] K. Pfab and M. Rothering, "Towards improved research methodologies for industrial AI: a case study of false call reduction," arXiv:2506.14521 [cs.LG], Jun. 2025. Source code and run logs: `https://gitlab.com/research9841542/FCR_1`.

[9] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" *J. Machine Learning Research*, vol. 15, no. 90, pp. 3133–3181, 2014.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022.

[11] R. Shwartz-Ziv and A. Armon, "Tabular data: deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022, doi: 10.1016/j.inffus.2021.11.011.

[12] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: a survey," *IEEE Trans. Neural Networks and Learning Systems*, vol. 35, no. 6, pp. 7499–7519, 2024, doi: 10.1109/TNNLS.2022.3229161.

[13] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[14] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, no. 8045, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[16] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Dept. of Statistics, University of California, Berkeley, Tech. Rep. 666, 2004.

[17] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[18] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[19] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: 10.1109/TKDE.2008.239.

[20] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010.

[21] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012, doi: 10.1016/j.patcog.2011.06.019.

[22] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, Art. no. 44, pp. 1–37, 2014, doi: 10.1145/2523813.

[23] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: a review," *IEEE Trans. Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019, doi: 10.1109/TKDE.2018.2876857.

[24] J. C. Bauer, S. Trattnig, F. Vieltorf, and R. Daub, "Handling data drift in deep learning-based quality monitoring: evaluating calibration methods using the example of friction stir welding," *J. Intelligent Manufacturing*, vol. 37, no. 2, pp. 759–774, 2025, doi: 10.1007/s10845-025-02569-6.

[25] J. Shim, S. Cho, E. Kum, and S. Jeong, "Adaptive fault detection framework for recipe transition in semiconductor manufacturing," *Computers & Industrial Engineering*, vol. 161, Art. no. 107632, 2021, doi: 10.1016/j.cie.2021.107632.

[26] N. Jourdan, S. Sen, E. J. Husom, E. Garcia-Ceja, T. Biegel, and J. Metternich, "On the reliability of machine learning applications in manufacturing environments," in *NeurIPS 2021 Workshop on Distribution Shifts*, 2021, arXiv:2112.06986.

[27] P. Kadlec, R. Grbić, and B. Gabrys, "Review of adaptation mechanisms for data-driven soft sensors," *Computers & Chemical Engineering*, vol. 35, no. 1, pp. 1–24, 2011, doi: 10.1016/j.compchemeng.2010.07.034.

[28] X. Tong, Y. Feng, and J. J. Li, "Neyman-Pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, Art. no. eaao1659, 2018, doi: 10.1126/sciadv.aao1659.

[29] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002, doi: 10.1162/089976602753284446.

[30] Z. C. Lipton, Y.-X. Wang, and A. Smola, "Detecting and correcting for label shift with black box predictors," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, PMLR vol. 80, 2018, pp. 3122–3130.

[31] G. Widmer and M. Kubat, "Learning in the presence of concept drift and hidden contexts," *Machine Learning*, vol. 23, pp. 69–101, 1996, doi: 10.1007/BF00116900.

[32] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, Lecture Notes in Computer Science, vol. 3171. Berlin, Germany: Springer, 2004, pp. 286–295, doi: 10.1007/978-3-540-28645-5_29.

[33] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. 7th SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448, doi: 10.1137/1.9781611972771.42.

[34] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: an analysis and review," *Int. J. Forecasting*, vol. 16, no. 4, pp. 437–450, 2000, doi: 10.1016/S0169-2070(00)00065-0.

[35] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE Trans. Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643.

[36] M. Gao, J. M. Perathoner, A. L. Bonin, S. Eulig, and G. Klesse, "Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification," arXiv:2605.07752 [cs.LG], 2026.